Professional Training in Cybersecurity I

Module 1 — Session 2

The Python Networking Stack

Mohamed Gebril

Session Objective

By the end you can do each of these, on a socket you opened yourself:

Open & drive — a socket through its lifecycle

`bind`/`listen`/`accept` against `connect`/`send`/`recv`, for TCP and UDP

Frame — messages on a byte stream

a length prefix, the empty `recv` that ends a read, and the boundary a stream drops

Diagnose — a call that fails

refused against timed out, a deadline with `settimeout`, and `Address already in use`

Secure & scale — the same socket wrapped and multiplexed

an `SSLContext` and its two checks, one thread per client, a lock, one selector loop

Reach lower — past the headers the API writes for you

which header each kind of socket writes, and the privilege a raw socket costs

Session Agenda

Part 1 — The Socket Interface
the socket as a kernel object; creating one; the 5-tuple; where the API meets the packet

Part 2 — Stream and Datagram Sockets
the lifecycle; binding; message boundaries; a length prefix; the empty `recv`; datagrams

Part 3 — Socket Errors and Timeouts
the `OSError` family; `settimeout`; refused against timed out; `SO_REUSEADDR`

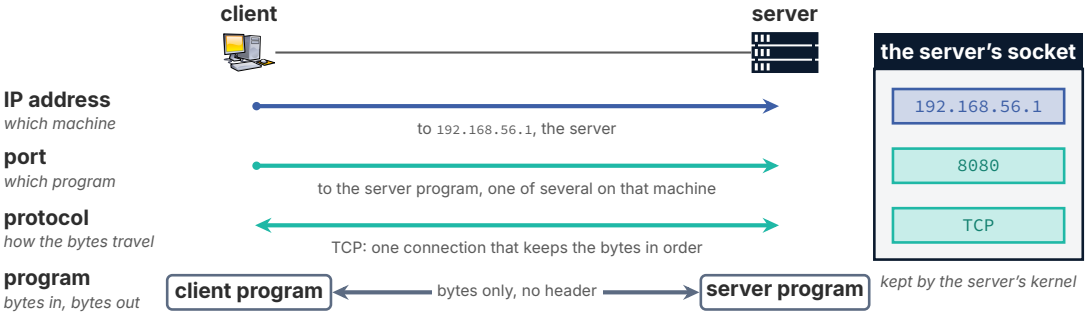
Part 4 — The Secure Socket: TLS
the plaintext problem; one call around the socket; two certificate checks; what stays visible

Part 5 — The Concurrent Socket
one blocking call per client; threads; a shared value and a lock; one thread over many sockets

Part 6 — Raw Sockets
which header each kind of socket writes, and the privilege boundary

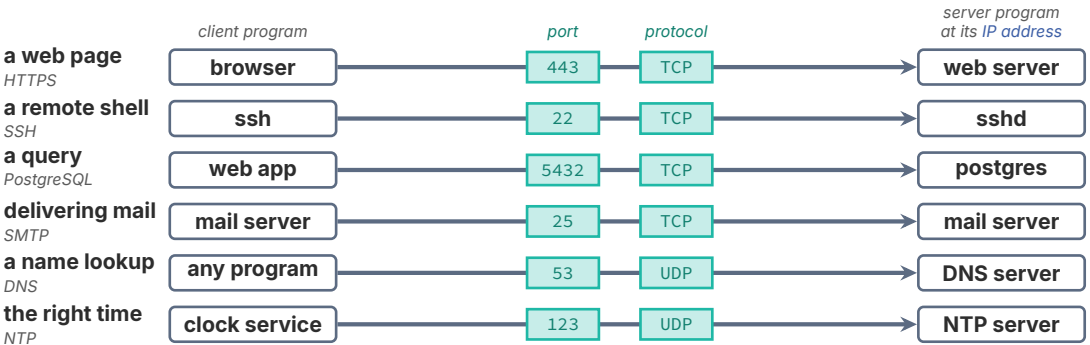
The Socket Interface

The Problem: Two Programs Need Three Settings



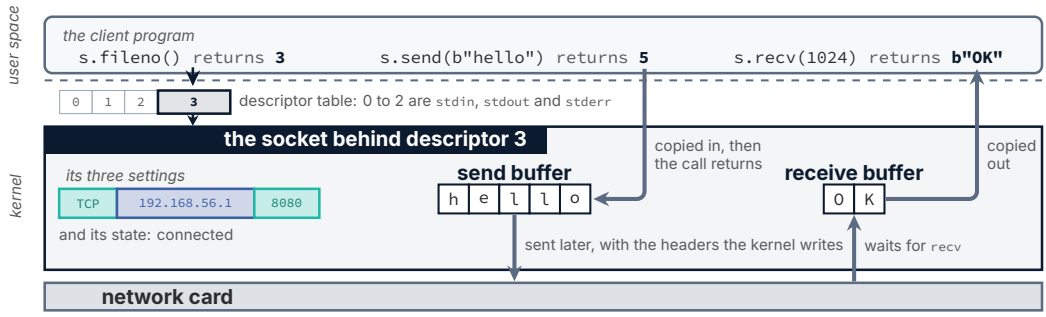
- A client program must reach a server program on another machine, and three settings come first.
- An **IP address** names the machine, a **port** the program on it, and the **protocol** how the bytes travel.
- The **kernel**, the core of the operating system, owns the network card and writes every header.
- A **socket** is the kernel's object that holds all three for one program, which only reads and writes bytes.
- The rest of this session is this one object in other forms: secured, concurrent and raw.

Real Programs: Every Network Service Holds a Socket



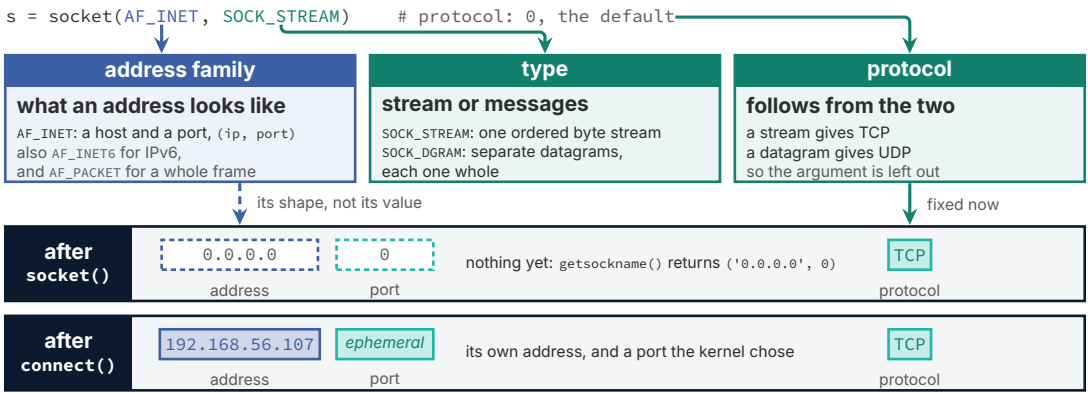
- Each row joins two machines, and a name lookup gives the client the server's **IP address**.
- Ports are registered with IANA, so a client knows where to connect: 0 to 1023 are **well-known**.
- An attacker reads the same list: a machine answering on port 22 is most likely running an SSH server.
- DNS falls back to TCP for a large answer, and HTTP/3 carries the web over UDP.

The Socket Object: Kernel State Behind a File Descriptor



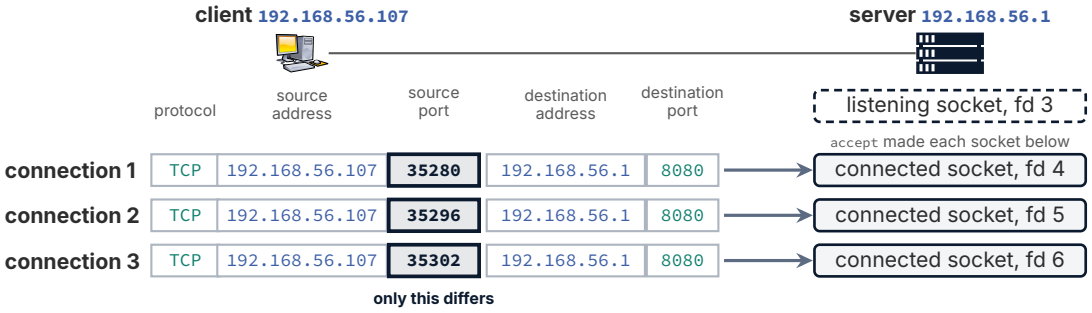
- Your program holds only a small integer, a **file descriptor**, and the kernel maps it to the socket, where all the state lives.
- `send` copies bytes into the **send buffer** and returns, and the kernel transmits them later: a returned `send` is not a delivery.
- Arriving bytes wait in the **receive buffer** until `recv` copies them out, so sends and receives need not line up one for one.

Creating a Socket: Family, Type and Protocol



- An **address family** fixes the shape of an address: in `AF_INET`, the IPv4 family, a host and a port.
- The **type** chooses one ordered stream or separate messages, and the protocol follows, so it stays `0`.
- A new socket has its protocol but **no address yet**: `connect` gives a client one, and `bind` gives a server one.

The 5-Tuple: What Identifies a Connection



- **Five values** identify a connection: the protocol, the source address and port, and the destination address and port.
- Three connections from one client to one service share four of them, so the **source port** keeps them apart: the kernel picks an unused one for each.
- On the server, each connection has **its own socket**, and the listening socket only takes in new ones.

Inspecting Live Sockets with `ss`

Local: this machine's end — Peer: the other end — the program, and the descriptor it holds

`$ ss -tanp sport = :8080` *on the server* — `-t` TCP, `-a` every state, `-n` numbers, `-p` the program

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	5	0.0.0.0:8080	0.0.0.0:*	users:(("python3",pid=46098,**fd=3**))
ESTAB	0	0	192.168.56.1:8080	192.168.56.107:35280	users:(("python3",pid=46098,**fd=4**))
ESTAB	0	0	192.168.56.1:8080	192.168.56.107:35302	users:(("python3",pid=46098,**fd=6**))
ESTAB	0	0	192.168.56.1:8080	192.168.56.107:35296	users:(("python3",pid=46098,**fd=5**))

listening: no peer yet

one socket for each connection

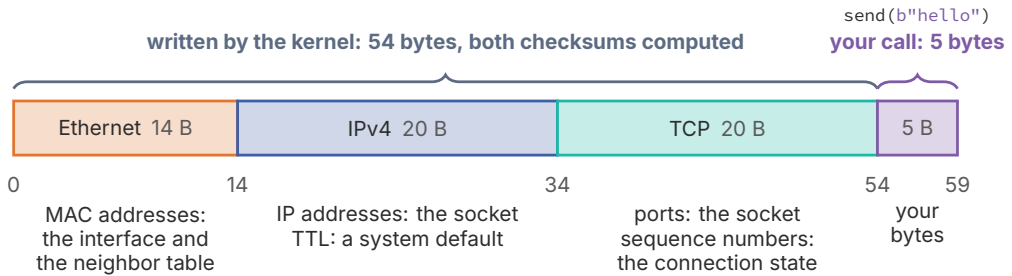
`$ ss -tnp dport = :8080` *on the client*

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
ESTAB	0	0	192.168.56.107:35302	192.168.56.1:8080	users:(("python3",pid=46099,**fd=5**))
ESTAB	0	0	192.168.56.107:35280	192.168.56.1:8080	users:(("python3",pid=46099,**fd=3**))
ESTAB	0	0	192.168.56.107:35296	192.168.56.1:8080	users:(("python3",pid=46099,**fd=4**))

the same three, ends swapped

- `ss` lists the **sockets of the machine it runs on**, and each machine lists **its own end as Local**.
- The **listening socket** has no peer yet (`0.0.0.0:*`), and each accepted connection is its own `ESTAB` row.
- The **Process** column names the program and its **descriptor**, and each machine numbers its own.

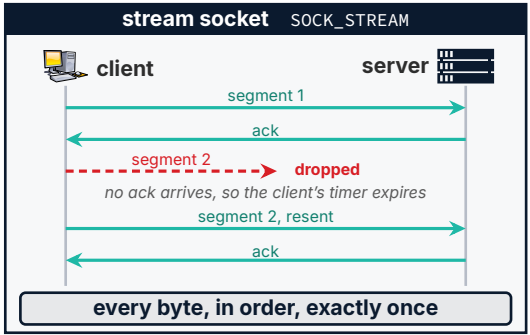
Where the API Stops and the Packet Begins

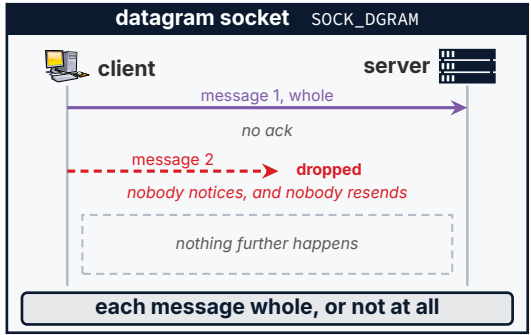


- Calling `send(b"hello")` puts five bytes on the wire and **fifty-four more that you never wrote**.
- The kernel fills each header from its own source: the **socket** for addresses and ports, the **connection** for sequence numbers, and the **neighbor table**, which maps an IP address on the link to its MAC.
- That is **the bargain of the socket API**: you never compute a checksum, pick a sequence number or resolve a MAC address.
- Taking that control back is what a **raw socket** is for, later in this session.

Stream and Datagram Sockets

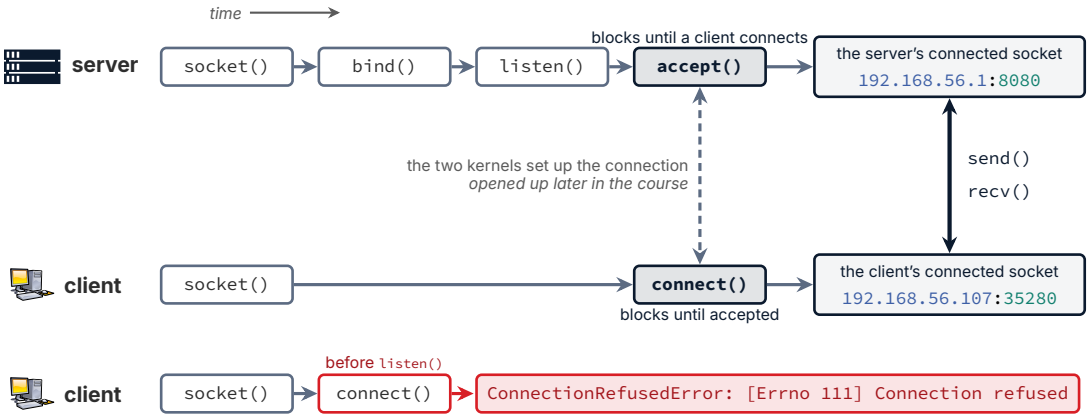
Why Two Kinds of Socket Exist





- Both kinds of socket run on the **same unreliable network**, which drops, reorders and duplicates packets.
- A **stream socket** numbers its bytes, waits for each acknowledgment and resends what is lost, and it pays in **latency and in state** the kernel keeps.
- A **datagram socket** sends each message once and **keeps it whole**, and promises nothing else.
- Choosing a type means choosing the promise to give up: **order and delivery**, or **message boundaries**.

The Connection Lifecycle: Two Sides, Different Calls



- The two sides **run different calls**: a server binds, listens and waits in `accept`, a client calls `connect`.
- `listen` must come **before** `connect`: a connection to a port where nothing listens is **refused**, not queued.
- `accept` and `connect` **block until the connection exists**, and then each side holds a connected socket.

The Two Skeletons, Side by Side

TCP server

```
s = socket(AF_INET, SOCK_STREAM)
s.bind(("0.0.0.0", 8080))
s.listen(5)
conn, peer = s.accept()
data = conn.recv(1024)  # b'hello'
conn.send(b"OK")
conn.close()
```

UDP server

```
s = socket(AF_INET, SOCK_DGRAM)
s.bind(("0.0.0.0", 9999))
```

no `listen()`: nothing to queue

no `accept()`: no connection to hand back

```
data, peer = s.recvfrom(1024)  # b'hello'
s.sendto(b"OK", peer)
```

no per-client socket to close

TCP client

```
s = socket(AF_INET, SOCK_STREAM)
s.connect(("192.168.56.1", 8080))
s.send(b"hello")
reply = s.recv(1024)  # b'OK'
```

UDP client

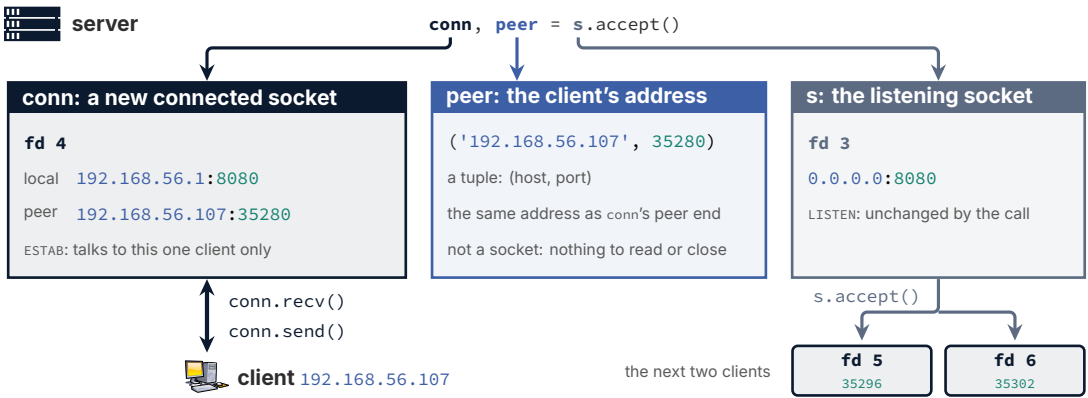
```
s = socket(AF_INET, SOCK_DGRAM)
```

no `connect()`: the address rides on every send

```
s.sendto(b"hello", ("192.168.56.1", 9999))
reply, addr = s.recvfrom(1024)  # b'OK'
```

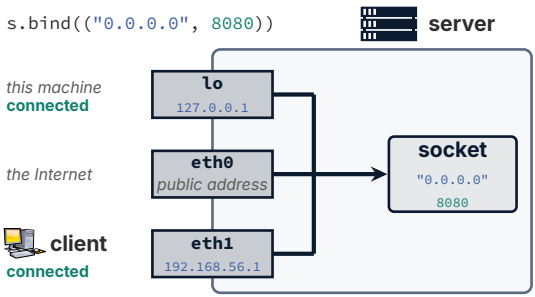
- Read the panels **across**: each line missing on the datagram side exists only to set up a connection.
- A datagram server keeps **one socket for all its clients**, so each `sendto` names the address it replies to.
- This code is **deliberately naive**: one exchange, no framing, no error handling, and the next frames break it.

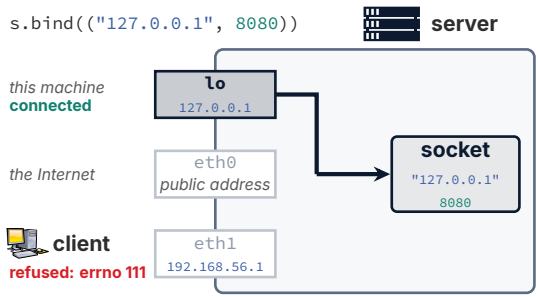
What `accept` Returns: A Second Socket



- `accept` returns a **pair**: `conn`, a new socket connected to one client, and `peer`, that client's address.
- The listening socket `s` is **never consumed**: it stays in `LISTEN`, and each later `accept` builds the next socket.
- The server talks to the client **through `conn`**, never through `s`, and closes `conn` when that client is done.

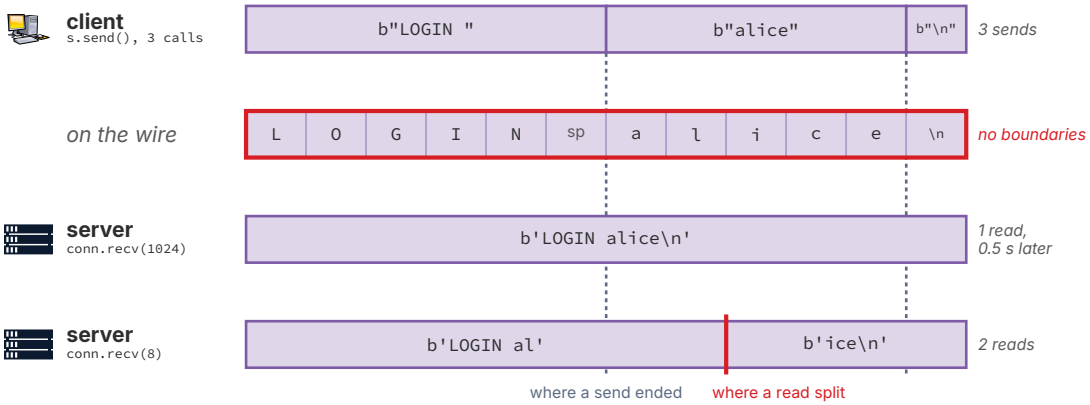
Binding: One Address or Every Interface





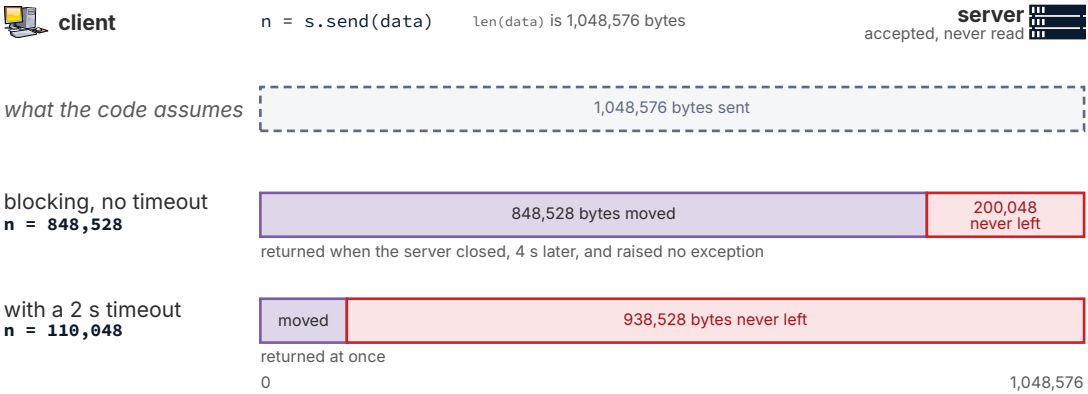
- An **interface** is one network attachment with its own address, and a host has several.
- In `AF_INET`, `0.0.0.0` is the **wildcard**, every interface. `127.0.0.1` is **loopback**, only this machine.
- Binding the wildcard publishes to **every network the host touches**, not only the one you meant.
- Bind the **narrowest address that reaches your peers**. In `AF_INET6` the same pair is written `::` and `::1`.

Where Message Boundaries Are Lost



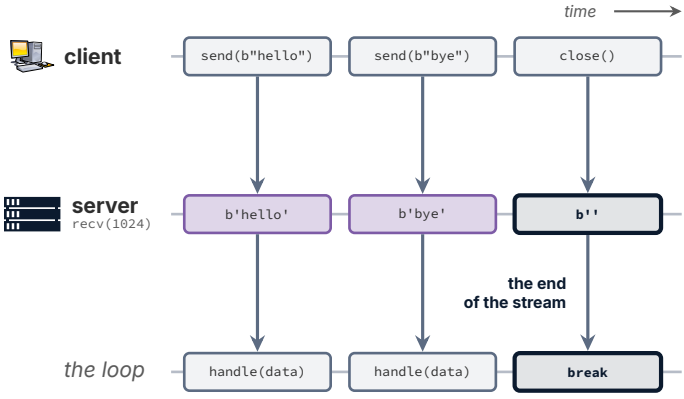
- A stream carries **bytes, not messages**: three sends became one run that records no boundaries.
- `recv` returns **what has arrived, up to the size asked**: `recv(1024)` took all 12 bytes, `recv(8)` split a word.
- On one machine small sends **usually arrive together**, which hides code that expects one `recv` per `send`.

What `send` Returns: A Byte Count



- `send` returns **the number of bytes it moved**, which can be fewer than offered, with no exception raised.
- Calling `send` once **assumes every byte left**: small messages usually do, so the bug hides in testing.
- A correct sender **loops on the returned count** until nothing is left, which is what `sendall` does for you.

An Empty `recv`: The Peer Has Closed



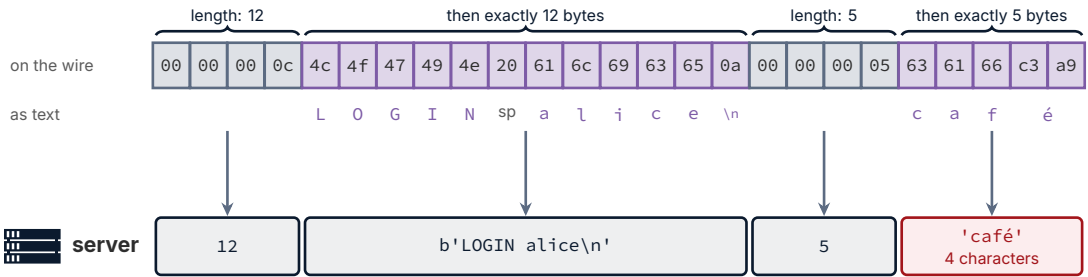
the receive loop

```
1  while True:
2      data = conn.recv(1024)
3      if not data:
4          break
5      handle(data)
```

- `recv` returns what has arrived, and **empty bytes, `b''`, only when the peer has closed** its side.
- So a receive loop **ends on `b''`**, never on a short read, which only means more bytes are on their way.
- A loop without that check **spins forever**, because every later `recv` returns `b''` at once.

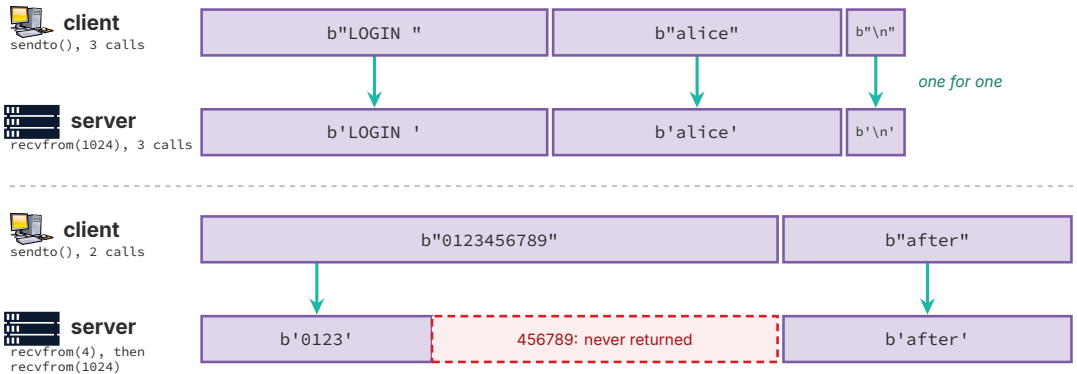
Restoring Boundaries with a Length Prefix

client `sock.sendall(struct.pack("!I", len(payload)) + payload)` for each message



- If the stream drops boundaries, **put them in the bytes**: a fixed 4-byte length before every message.
- The reader takes **two steps**: exactly four bytes for the length, then exactly that many, and **both must loop**.
- The prefix counts **encoded bytes, never characters**: a prefix of 4 for `"café"` would leave one byte behind.
- A delimiter byte or fixed-size records work too: each **puts back the boundary** the stream dropped.

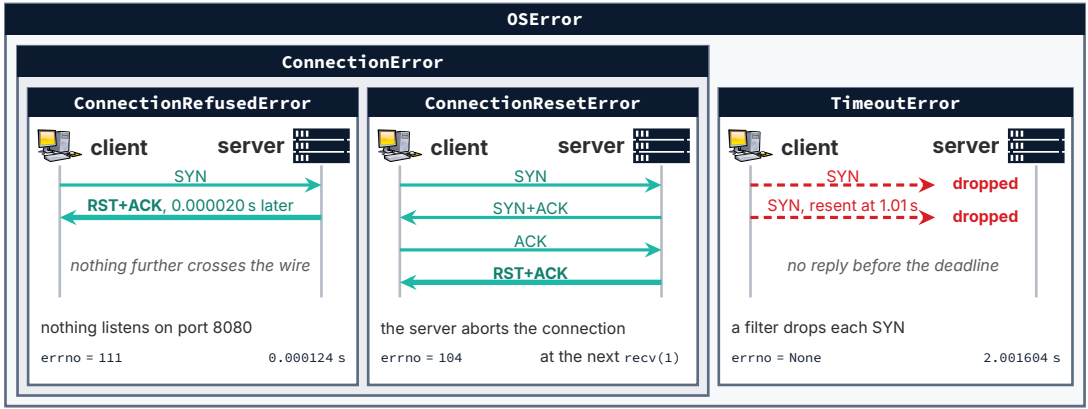
Datagrams Keep Their Boundaries



- A datagram socket delivers **messages, not bytes**: one `sendto` becomes one `recvfrom`, boundary intact.
- So datagram code **needs no framing**: the datagram header already carries the length, as a prefix would.
- A read that is too small **loses the rest of that datagram**, so size the buffer for the largest message.
- Nothing else is promised: datagrams can **arrive out of order, twice, or not at all**, and nobody is told.

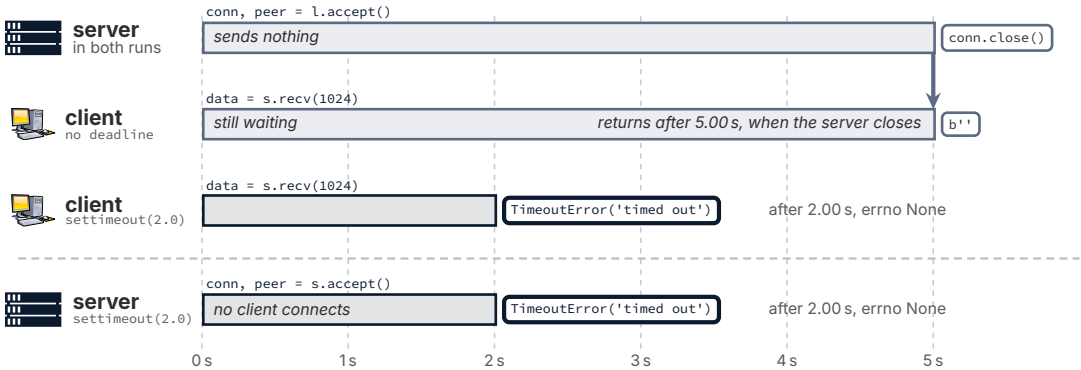
Socket Errors and Timeouts

Socket Failures: Exceptions Describe Different Events



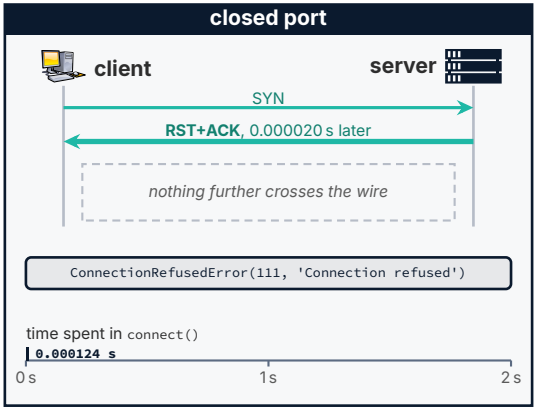
- A failing socket call raises an **exception**, a subclass of `OSError`, and the subclass names what came back.
- A **reset** refuses an attempt or aborts a connection, and one `ConnectionError` handler catches both.
- **Silence** brings no answer, so only the client's own deadline ends the wait, and `TimeoutError` has no `errno`.

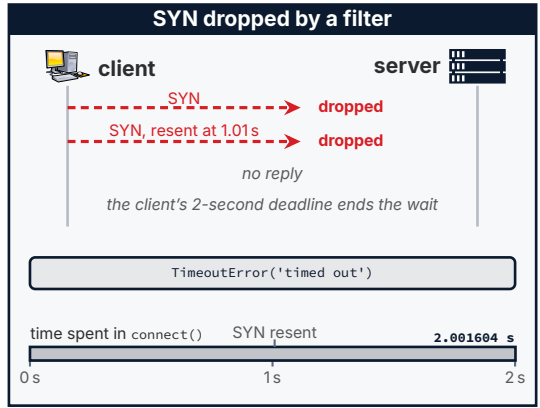
`settimeout`: A Deadline on Every Blocking Call



- By default a blocking call has **no deadline**: it returns only when the peer sends or closes.
- `settimeout(n)` gives **every later blocking call on that socket** a deadline of `n` seconds, `accept` included.
- When the deadline passes, the call raises `TimeoutError`, so the program **can report it, retry or move on**.
- A tool with no deadline can be **stalled by one peer that never answers**, so every tool in this course sets one.

Connection Failure: Refusal or an Expired Deadline





- A closed port **answers at once**: its reset made `connect` raise `ConnectionRefusedError` in 0.000124 seconds.
- A dropped SYN **gets no answer**: the kernel resends it, and only the client's deadline ends the attempt.
- A reset is **evidence of a live host**, and silence cannot tell a filter from loss.

Address Already in Use: `TIME_WAIT` and `SO_REUSEADDR`



server
first run

`conn.close()` first, then exit




```
$ ss -tano sport = :8080        on the server, after the first run
State       Recv-Q  Send-Q     Local Address:Port   Peer Address:Port        Process
TIME-WAIT   0       0          192.168.56.1:8080   192.168.56.107:48684     timer:(timewait,59sec,0)
```

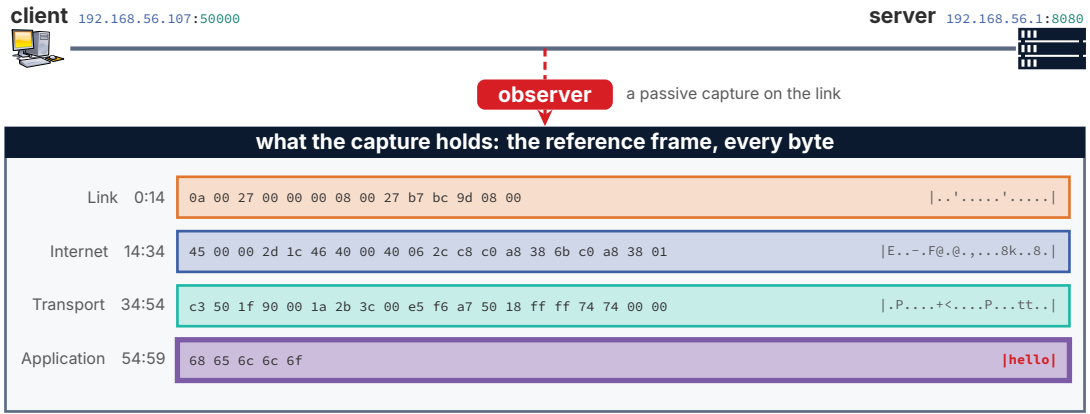
the option: `s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)` before `bind`

	first run: option not set	**first run: option set**
restart: option not set	`OSError: [Errno 98] Address already in use`	`OSError: [Errno 98] Address already in use`
restart: option set	`OSError: [Errno 98] Address already in use`	**`bind ok`**

- The side that **closes first** keeps the connection in `TIME_WAIT` for 60 seconds, until late segments expire.
- A server restarted inside that window cannot `bind` its port, and the call raises `OSError` with errno 98.
- A **socket option** is a per-socket setting the kernel reads: `SO_REUSEADDR` works only if **every run sets it**.

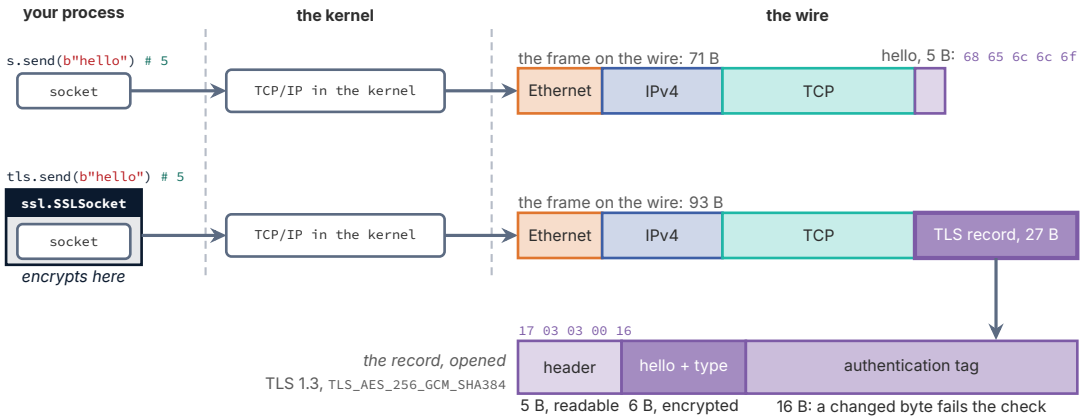
The Secure Socket: TLS

The Plaintext Problem



- A plain socket's bytes cross a path **neither end controls**, and anyone who can capture on it reads them.
- Reading needs **no attack and leaves no trace**: a passive capture changes nothing either end can see.
- **TLS** (Transport Layer Security) encrypts the byte stream, reached **through the socket you already have**.

What TLS Wraps: One Call Around the Socket



- `wrap_socket` returns an object that **behaves like the socket it was given**: the same `send` and `recv`.
- The encryption happens **inside your process**, so the switches and routers in between need no change.
- Before the first `send`, a **TLS handshake** checks the server's **certificate** and agrees on the keys.

The TLS Skeleton: What Changes, and What Does Not

TLS server

```
ctx = ssl.SSLContext(
    ssl.PROTOCOL_TLS_SERVER)
ctx.load_cert_chain("peer.crt", "peer.key")
s = socket(AF_INET, SOCK_STREAM)
s.bind(("0.0.0.0", 8443))
s.listen(5)
conn, peer = s.accept()
tls = ctx.wrap_socket(conn,
    server_side=True)
data = tls.recv(1024)  # b'hello'
tls.send(b"OK")
tls.close()
```

TLS client

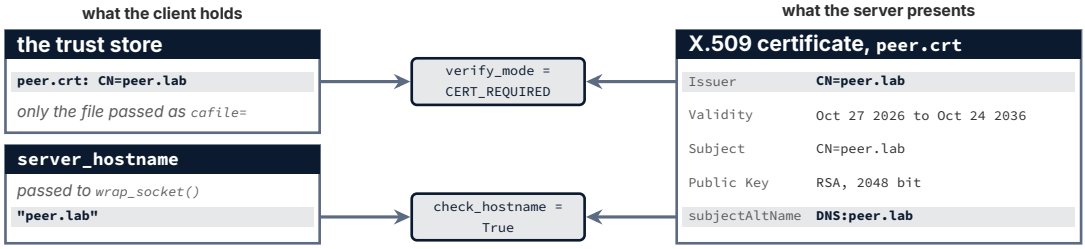
```
ctx = ssl.create_default_context(
    cafile="peer.crt")
```

no certificate: only the server proves who it is

```
s = socket(AF_INET, SOCK_STREAM)
tls = ctx.wrap_socket(s,
    server_hostname="peer.lab")
tls.connect(("192.168.56.1", 8443))
tls.send(b"hello")
reply = tls.recv(1024)  # b'OK'
tls.close()
```

- An `SSLContext` is **local configuration, not a connection**: the certificate, the key and what to verify.
- Only the server **presents a certificate by default**, and **mutual TLS (mTLS)** adds one from the client.
- `wrap_socket` is **per connection**: the server wraps what `accept` returned, **never the listener**.

Verification: Two Checks on One Certificate

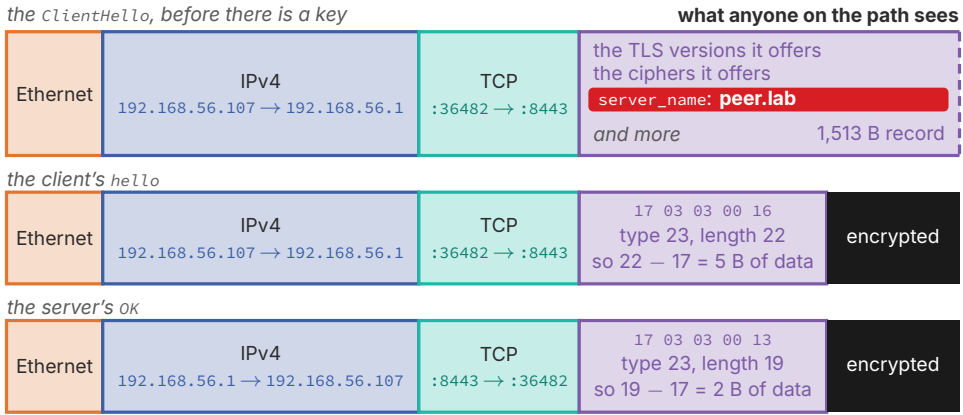


no cafile=: SSLCertVerificationError
verify_message = 'self-signed certificate'
verify_code = 18

server_hostname="other.lab": SSLCertVerificationError
"Hostname mismatch, certificate is not valid for 'other.lab'."
verify_code = 62

- A certificate **binds a name to a public key**, signed by an issuer, and the client verifies that binding.
- `create_default_context` turns on two checks: **trust** in the issuer, and the **name** against `server_hostname`.
- Either failure raises in `connect`, **before any of your bytes move**.
- Turning either check off makes a **substituted certificate invisible to the client**.

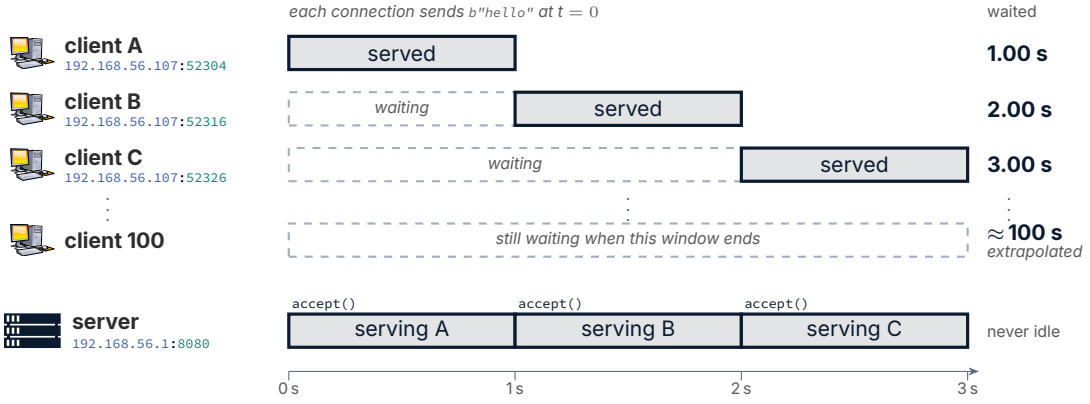
What TLS Still Does Not Hide



- TLS encrypts **the payload and nothing beneath it**: every header stays readable, so the path can deliver it.
- An observer still learns **who talks to whom, on which ports, how much and when**, without reading content.
- The **hostname** in `server_hostname` crosses **in the clear**, so the server can choose its certificate.

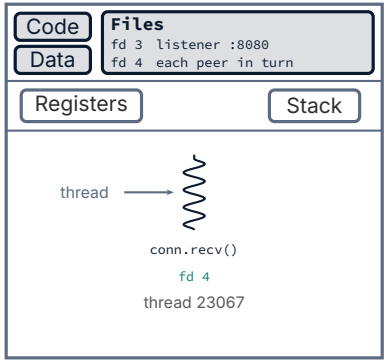
The Concurrent Socket

One Blocking Call, One Client

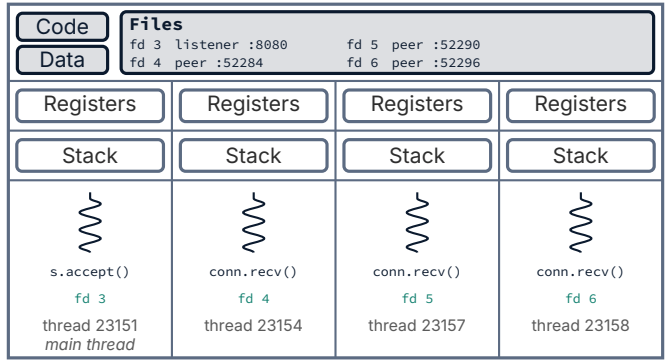


- A **blocking call** returns only when its work is finished, and its caller does nothing else until then.
- A server built from blocking calls therefore serves **one client from start to finish** before it looks at the next.
- As clients arrive the wait grows. The server never pauses, yet **each client waits for every client ahead of it**.

One Process, Many Threads



Single-threaded, pid 23067



Multi-threaded, pid 23151

- A **thread** is an independent path of execution inside a process, with its own registers and stack.
- Code, data and open descriptors are **shared by every thread**: fds 3 to 6 sit in one process's table.
- Each thread holds its own call in progress, so **each can block independently** of the others.

One Thread, One Client

threaded server: three clients

```
client A 192.168.56.107:52284 got b'OK' after 1.00 s
client B 192.168.56.107:52290 got b'OK' after 1.00 s
client C 192.168.56.107:52296 got b'OK' after 1.00 s
```

TCP server

no handler function: the calls run inline

```
1 s = socket(AF_INET, SOCK_STREAM)
2 s.bind(("0.0.0.0", 8080))
3 s.listen(5)
```

no loop: `accept` runs once

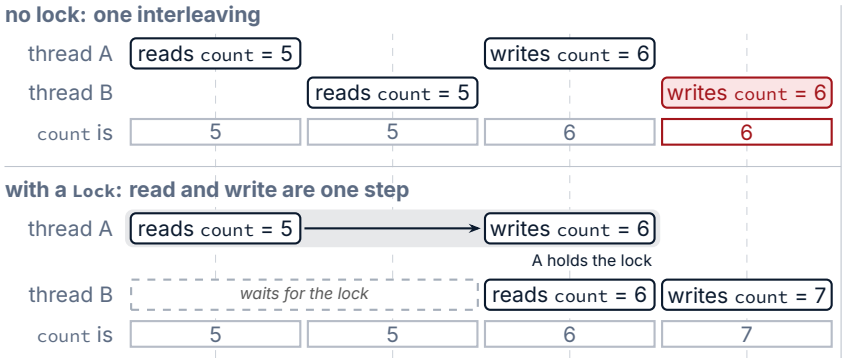
```
4 conn, peer = s.accept()
5 data = conn.recv(1024)
6 conn.send(b"OK")
7 conn.close()
```

threaded server

```
1  def handle(conn):
2      data = conn.recv(1024)  # b'hello'
3      time.sleep(1.0)  # one second of work
4      conn.send(b"OK")
5      conn.close()
6  s = socket(AF_INET, SOCK_STREAM)
7  s.bind(("0.0.0.0", 8080))
8  s.listen(5)
9  while True:
10     conn, peer = s.accept()
11     Thread(target=handle,
12            args=(conn,)).start()
```

- A **handler** is the function that serves one connection, because a thread must be given a function to run.
- `accept` moves into a **loop**, and each connection starts a thread. The calls are **unchanged**.
- The blocking limit is **multiplied**, not gone: in the lab, the same three clients each waited 1.00 s.

Two Threads, One Value: One Update Is Overwritten



two threads, 400,000 each

492,292 to 601,772

of 800,000 expected
three runs, each different

```
count = bump(count)
```

800,000

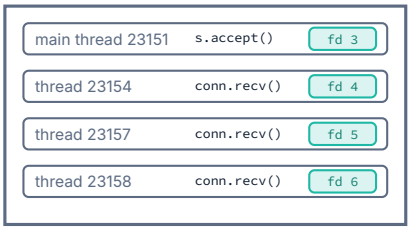
of 800,000 expected
three runs, every time

```
with lock:
    count = bump(count)
```

- A **lost update**: two threads read one value, both compute from it, and one write erases the other.
- It loses a **different amount every run**, with no error, so a test that passes once proves nothing.
- A `Lock` lets **one thread at a time** read, compute and write, so the three act as one step.

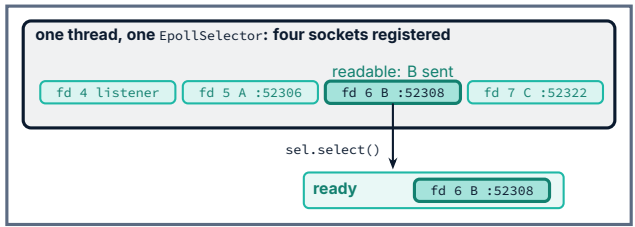
One Thread Waits on Many Sockets

one thread per socket



four threads, each blocked in its own call

one thread over many sockets



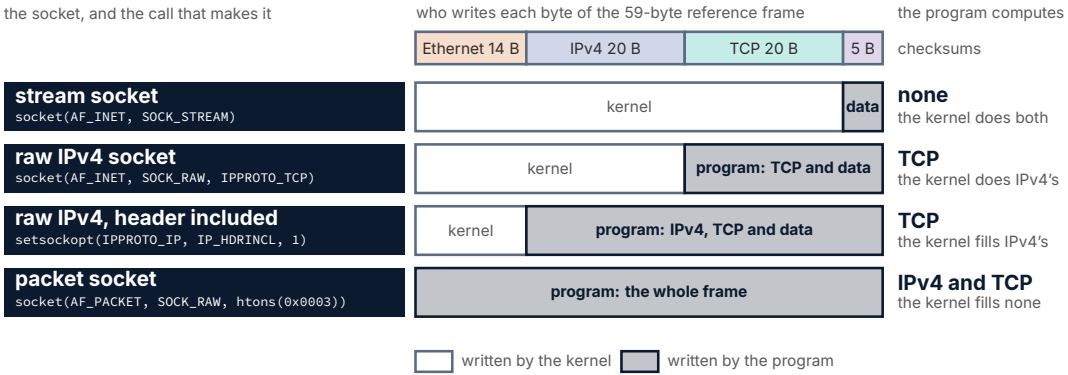
one call returned 1 ready of 4 registered

```
selector: EpollSelector
select() returned 1 ready of 4 registered: fd 6 ('192.168.56.107', 52308)
  fd 6 recv returned b'hello'
```

- **I/O multiplexing** is one thread waiting on many sockets, in place of a blocked thread per socket.
- A socket is **readable** when bytes have arrived, and **writable** when its send buffer has room.
- A **selector** is the set of sockets the kernel watches. One call returns only the ready ones.
- A slow client can **stall a serial server**. Threads and selectors let other clients progress.

Raw Sockets

Raw Sockets: Moving the Header Boundary



- A **raw socket** lets the program write headers that a stream socket leaves to the kernel.
- Each lower interface moves the **boundary between program bytes and kernel bytes** down the stack.
- A header the program writes is **its own to get right**, checksum included.
- No raw socket provides TCP's reliable stream: that stays with `SOCK_STREAM`.

Raw IPv4: Constructing an ICMP Echo Request

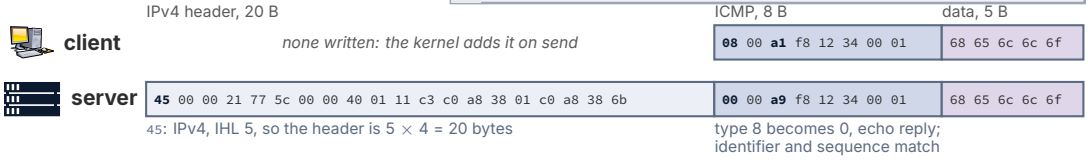
ICMP echo request

type **8**	code **0**	checksum **0xa1f8**
identifier **0x1234**		sequence number **1**
data: **hello** (5 bytes)		

8-byte header plus data: 13 bytes, checksum over all 13 validates to 0

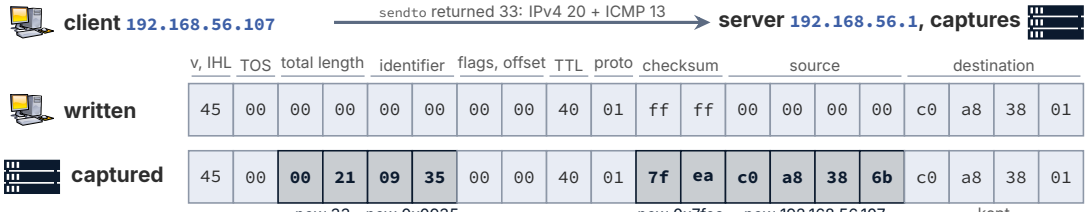
ICMP echo, client

```
1 hdr = struct.pack("!BBHHH", 8, 0, 0, 0x1234, 1)
2 check = checksum(hdr + b"hello")
3 hdr = hdr[:2] + struct.pack("!H", check) + hdr[4:]
4 s = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)
5 s.settimeout(2)
6 s.sendto(hdr + b"hello", (peer, 0))
7 reply, addr = s.recvfrom(65535)  # 33 bytes
```



- **ICMP** carries Internet-layer control messages: an echo request asks the server to return its data.
- With `SOCK_RAW` and `IPPROTO_ICMP`, the program writes the ICMP message, **checksum included**.
- The reply arrives with the kernel's IPv4 header in front, so **IHL locates the ICMP reply**.

IP_HDRINCL: Supplying the IPv4 Header



raw ICMP socket, IPv4 header included

```
ip = struct.pack("!BBHHHBBH4s4s", 0x45, 0, 0, 0, 0, 64, IPPROTO_ICMP, 0xffff,
                 b"\0"*4, inet_aton("192.168.56.1"))
s.setsockopt(IPPROTO_IP, IP_HDRINCL, 1)
s.sendto(ip + icmp, ("192.168.56.1", 0))  # 33
```

- With `IP_HDRINCL` set, the send buffer **starts with an IPv4 header** the program wrote.
- Linux still fills the length and checksum, and the source and identifier **when left at zero**.
- Choosing a source address does not prove you own it: the basis of **address spoofing**.

AF_PACKET: Sending a Complete Ethernet Frame



client, interface `eth1`

`send` returned 59: every byte the program's

server, captures on `vboxnet0`



Ethernet 14 B MACs, EtherType `0800`	IPv4 20 B checksum `0x2cc8`	TCP 20 B checksum `0x7474`	5 B `hello`

packet socket, client

```
1  s = socket(AF_PACKET, SOCK_RAW, htons(0x0003))  # ETH_P_ALL
2  s.bind((interface, 0))  # interface = "eth1"
3  sent = s.send(frame)  # 59
4  incoming, addr = s.recvfrom(65535)  # 47 B, EtherType 0800: an echo request from the server
```

server capture output

```
received bytes= 59
exact reference match= True
IPv4 checksum validation= 0 TCP checksum validation= 0
```

59 bytes left the client and 59 arrived:
on this virtual link neither buffer holds
Ethernet padding or the frame check sequence

- An `AF_PACKET` socket is bound to an interface (**interface binding**), not to a port.
- `htons(0x0003)` receives **every protocol** the interface carries, Ethernet header included.
- The program writes **every header and both checksums**, and the server got all 59 bytes unchanged.

Raw Access: The CAP_NET_RAW Privilege Boundary

process has CAP_NET_RAW

```
CapEff: 000001ffffffffff
CapBnd: 000001ffffffffff
```

`f` = 1111: bit 13, `CAP_NET_RAW`, is set

the same two calls:

`socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)`	**created**
`socket(AF_PACKET, SOCK_RAW, htons(0x0800))`	**created**

process lacks CAP_NET_RAW

```
CapEff: 000001ffffffdfff
CapBnd: 000001ffffffdfff
```

`d` = 1101: bit 13, `CAP_NET_RAW`, is cleared

the same two calls:

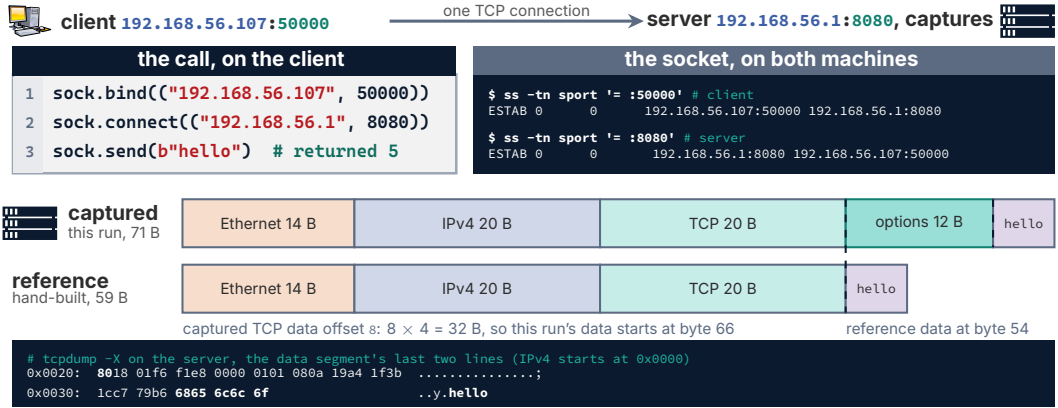
`socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)`	**PermissionError errno=1**
`socket(AF_PACKET, SOCK_RAW, htons(0x0800))`	**PermissionError errno=1**

the two runs

```
$ python3 lab.py --worker cap_test
$ setpriv --bounding-set=-net_raw --inh-caps=-net_raw --ambient-caps=-net_raw python3 lab.py --worker cap_test
# the second run removes CAP_NET_RAW from the bounding set, so the Python it starts never holds it
```

- A **capability** is one Linux privilege, held per thread: raw and packet sockets need `CAP_NET_RAW`.
- Without it `socket()` raises `PermissionError`: run only the lab's trusted program with `sudo`.
- Granting it to a Python interpreter grants it to **every script that interpreter runs**.

One Exchange: The Call, the Socket and the Wire



- Three views of **one connection**: the call's return value, the socket's endpoints, the bytes on the wire.
- `send` returning 5 means **accepted locally**. Only the server's capture shows the bytes crossed.
- This run's TCP header is 32 bytes: **read each header's length**, never assume fixed offsets.

Key Takeaways

- A **socket** is an endpoint the kernel keeps for one program. Its family, type and protocol fix what it is, and five values identify each connection.
- A stream keeps order and **drops message boundaries**. Put the boundaries back with a length prefix, loop on what `send` returns, and end a read loop on `b''`.
- Every blocking call needs a **deadline**, and a server needs `SO_REUSEADDR` from its first run, or a restart cannot bind.
- **TLS** is one call around the socket, and it protects only while both certificate checks stay on. Headers, sizes and the hostname still show.
- One blocking call serves one client. **Threads** multiply the callers, shared state then needs a **lock**, and a **selector** lets one thread serve many.
- **Raw sockets move the header boundary**. Supply and verify every header the chosen interface exposes, and keep raw access behind its privilege boundary.

How the Practical Guide Uses This

1. **Open the Practical Guide.** Run a client against a server on the other lab machine, then make it hold up: framing and the empty `recv`, deadlines and failed connections, TLS, threads and raw sockets.
2. **Record as you go.** Code, outputs and screenshots go into the **Submission worksheet** while you work, not at the end.
3. **Then the Home Task.** Apply the same method on your own. It has its own worksheet.

Keep the **Tooling Reference** (the socket calls, their exceptions and their options) and the **Troubleshooting FAQ** beside you throughout.

Open the Practical Guide

`Module1_Session2_Practical_Guide.md`

Module 1 — Session 2
The Python Networking Stack